EnvPilot: An Environment-Aware Coding Agent

Tingfeng Lan, Yusen Wu, Yi Deng

4/28/2026

Motivation

Proactive Grounding Works

- Fewer system crashes
- Significantly lower repair costs

The Reactive Paradigm Gap

- generate → execute → repair cycle
- Expensive under environment mismatch

Dynamic Runtime Reality

- API/Docs drift over time
- Static knowledge is insufficient

We need an agent—not just a checker—that decides when to probe, search, validate, and generate.

Background

Static LLMs, Dynamic Runtimes

Code LLMs
- Trained on static API corpora
- No access to runtime dependency state

Real-World Environments
- Version-specific APIs
- Rapid library evolution
- Cross-package constraints

Consequence
- Environment–model misalignment at inference time

Related Work

Version-Sensitive Code Generation
(e.g., Wu et al., ICSE 2026 – VersiBCB)

- Condition generation on library versions
- Static environment conditioning

API Knowledge Updating
(e.g., Wang et al., 2025 – ReCode)

- RL-based API knowledge updating
- Improve deprecated API handling

Execution-Grounded Debugging Paradigm
(e.g., SWE-bench, 2023–2025 evaluation line)

- Code generation evaluated via real execution
- Often implemented with generate–execute–repair loops
- Reactive, post-failure correction

Claim / Target Task

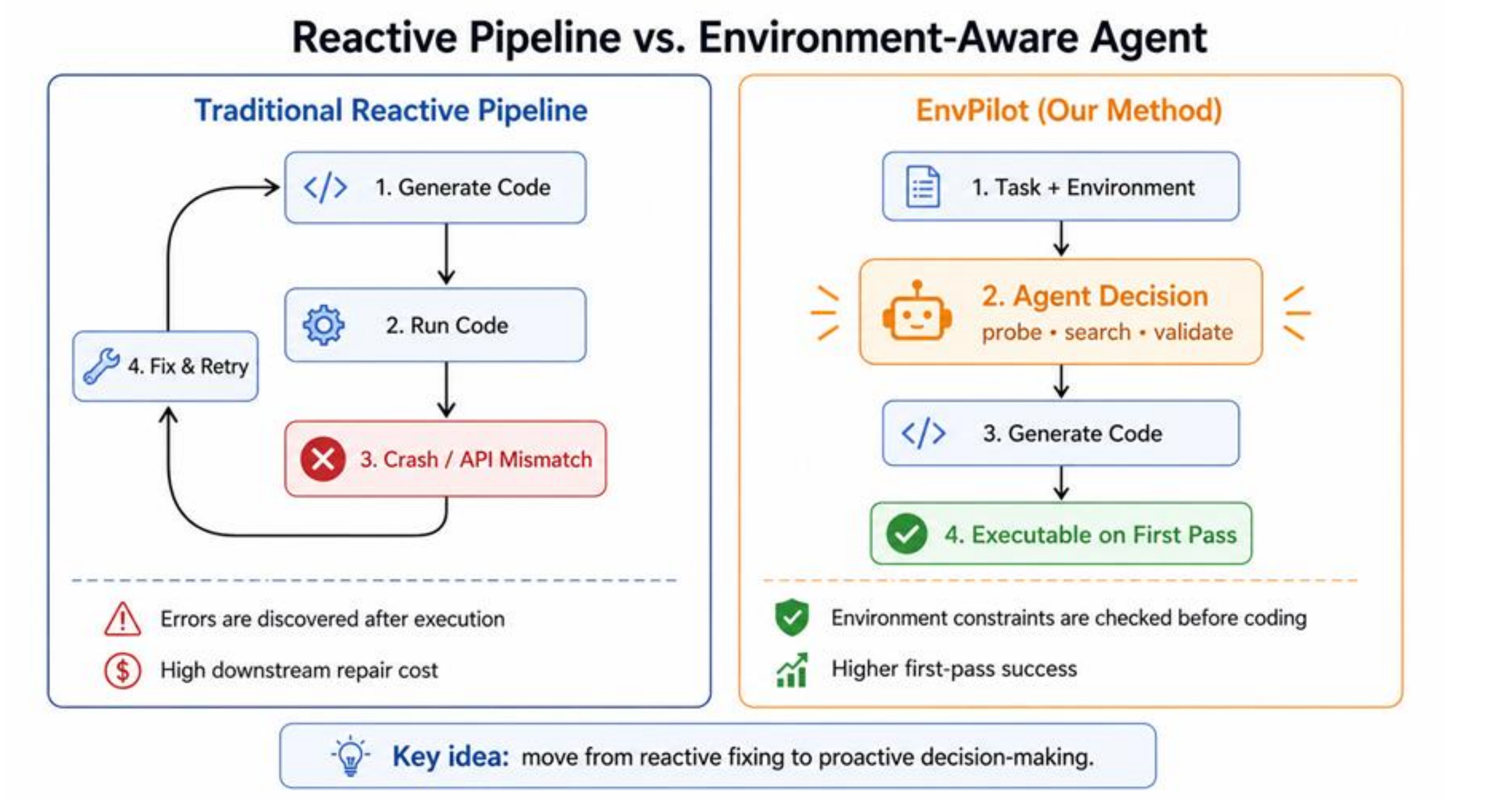
Enabling "Environment-First" Code Generation

Key Objectives:

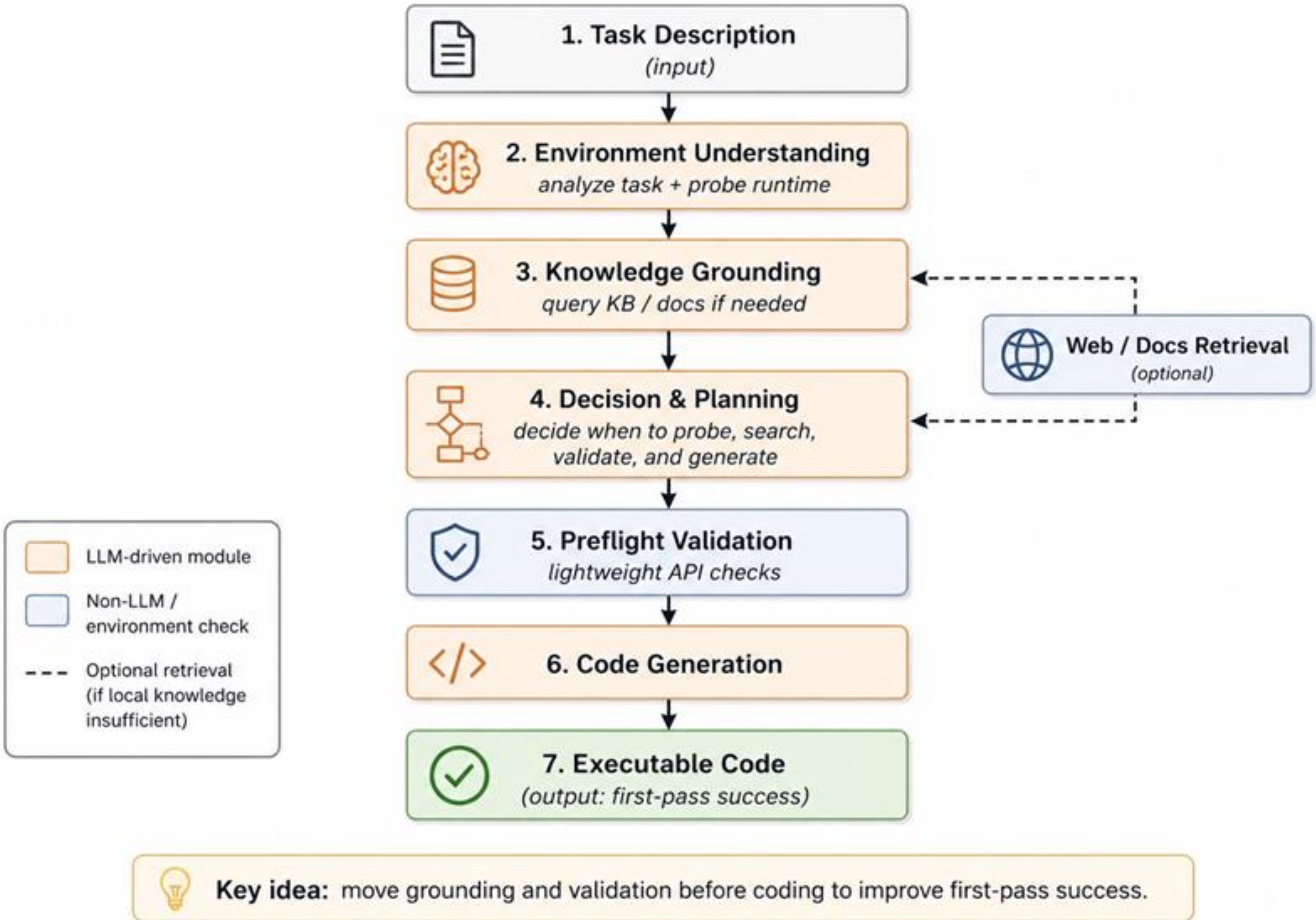
- Decide when to probe the environment
- Decide when to search or validate information
- Decide when to generate or repair code

Generate executable code that works in the target environment on the first attempt

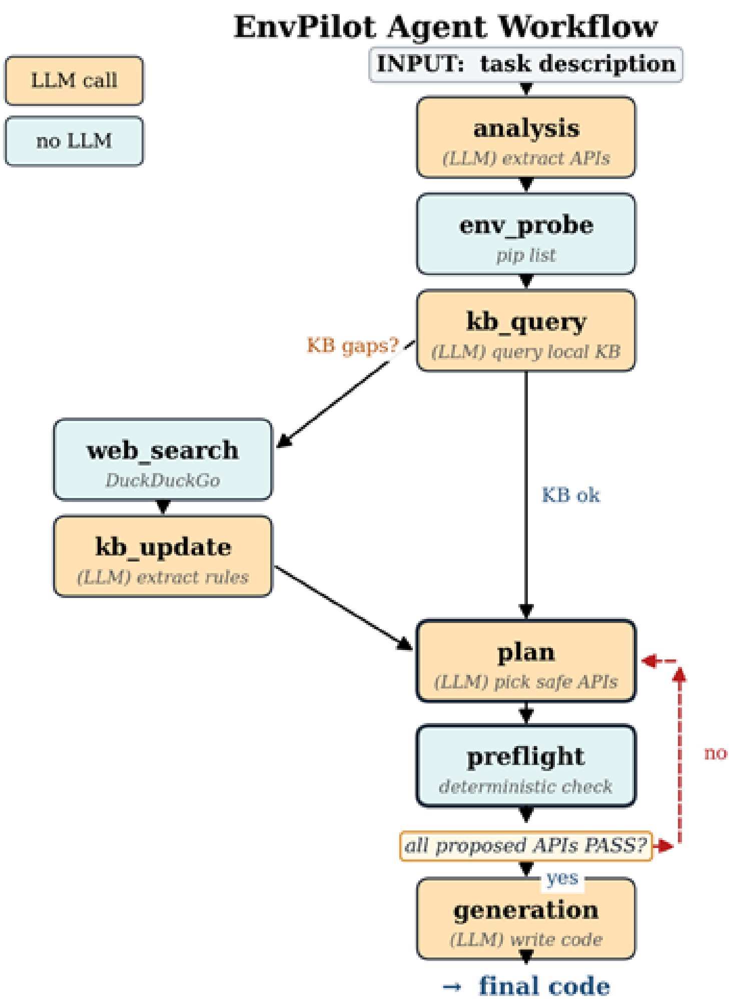
An Intuitive Figure Showing WHY Claim



Proposed Solution



Implementation



How to evaluate LLM agents?

01. Tasks

Task Setting:

- Environment-sensitive tasks
- API / version mismatch cases
- Held-out library/doc updates

Based on **BigCodeBench** with manually modified versions

02. Comparison Methods

Baseline:

- Direct generation (single LLM call)

Our method:

- EnvPilot (our agent)

03. Metrics

Effectiveness:

- First-pass success
- Final success
- Crash rate

Efficiency:

- Latency / tokens
- Tool calls
- Overhead vs. repair savings

What to evaluate for Agentic AI System?

Data Summary

Source

- Built on top of BigCodeBench (Zhuo et al. 2024)
- Each task provides a prompt, canonical solution, test cases, and entry point

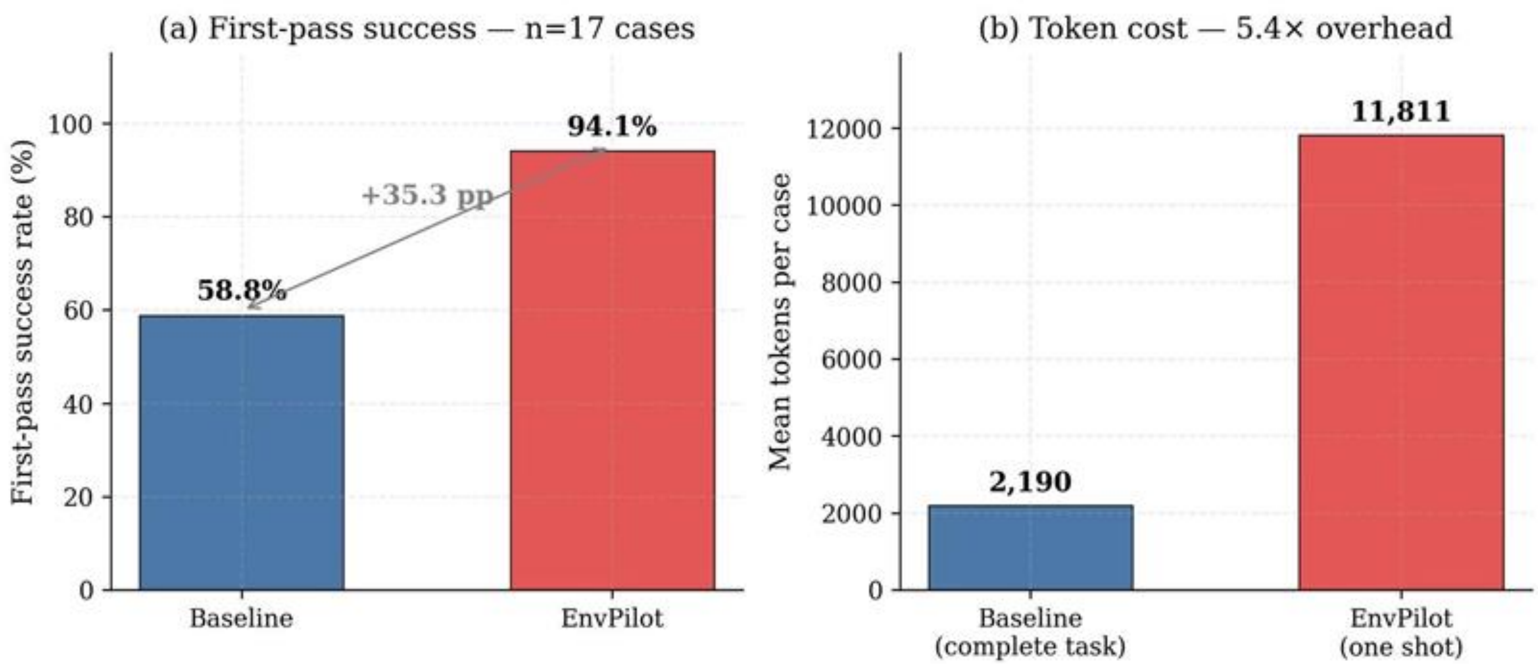
Construction Process

1. Start from a BigCodeBench task
2. Manually upgrade / downgrade library versions
3. Run the canonical solution in the modified environment
4. Keep the case if it fails due to API / version mismatch

Final Dataset

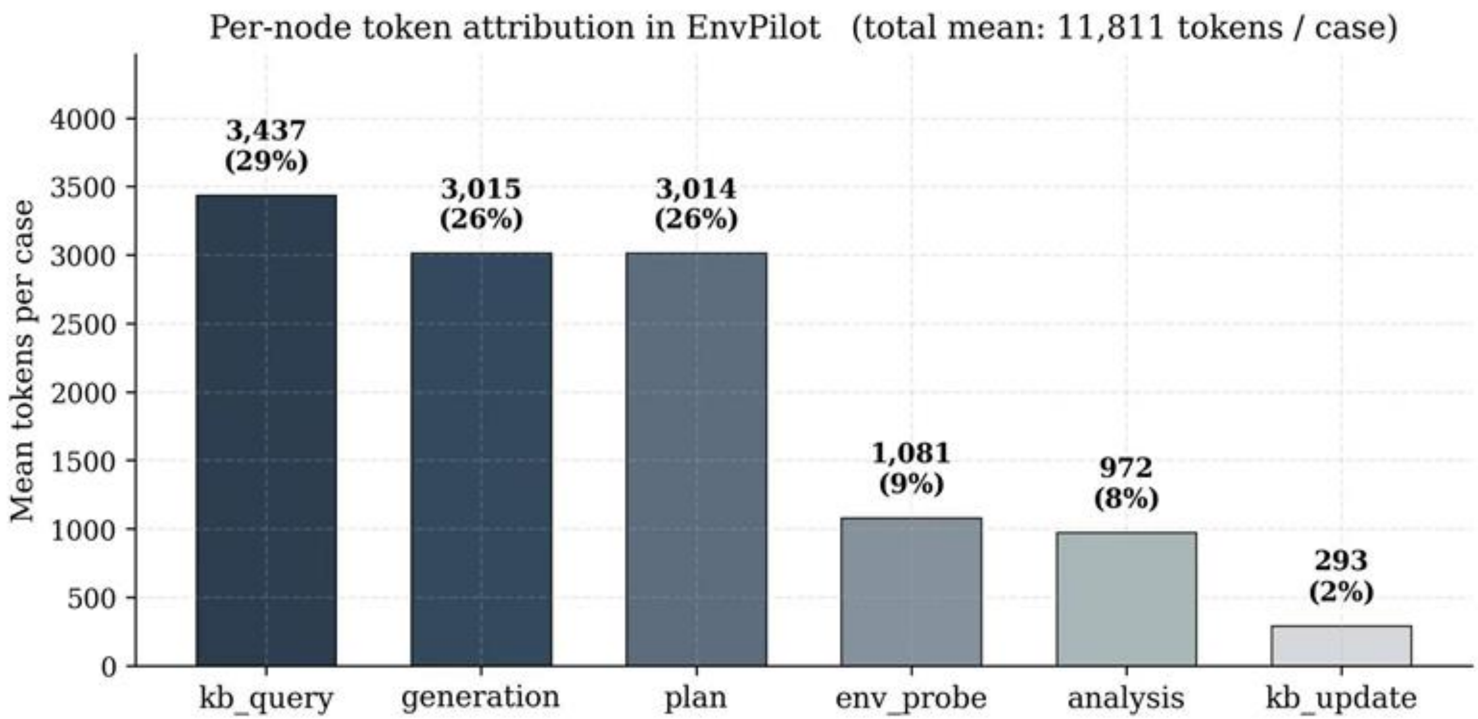
- 23 version-mismatch cases
- Environment itself is valid and installable
- Failure is caused by code–environment incompatibility

Experimental Results



- **+35.3 pp improvement** in first-pass success (58.8% → 94.1%)
- **Higher token cost** (5.4× compared to baseline)

Experimental Results



- **~80% of tokens** are spent on **reasoning and planning** (kb_query + plan + generation)
- **Environment probing is only ~9%** of total cost

Experimental Analysis

Main Finding:

EnvPilot trades higher upfront cost for much higher first-pass success.

Why it works:

- Proactive grounding prevents version-mismatch failures
- Preflight validation catches risky API assumptions before coding

Where the cost comes from:

- ~80% from reasoning and planning
- only ~9% from environment probing

Conclusion and Future Work

Environment-aware agents can **significantly improve** first-pass success

The cost increase mainly comes from **reasoning and planning**

Proactive grounding is **worth it when failures are expensive**

Future Work: Improve decision efficiency, reduce unnecessary KB queries, and scale evaluation to more libraries and real-world environments.

References

1. Wu, T., Chen, R., Du, W., Ma, S., Qi, G., Xing, Z., Khadivi, S., Periyathambi, R. and Haffari, G., 2026. Environment-Aware Code Generation: How far are We?. arXiv preprint arXiv:2601.12262.
2. Miao, C., Zou, H.P., Li, Y., Chen, Y., Wang, Y., Wang, F., Li, Y., Yang, W., He, B., Zhang, X. and Yu, D., 2025. Recode-h: A benchmark for research code development with interactive human feedback. *arXiv preprint arXiv:2510.06186*.
3. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. and Narasimhan, K., 2023. Swe-bench: Can language models resolve real-world github issues?. *arXiv preprint arXiv:2310.06770*.
4. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. and Press, O., 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, *37*, pp.50528-50652.
5. Yang, J., Jimenez, C.E., Zhang, A.L., Lieret, K., Yang, J., Wu, X., Press, O., Muennighoff, N., Synnaeve, G., Narasimhan, K.R. and Yang, D., 2024. Swe-bench multimodal: Do ai systems generalize to visual software domains?. *arXiv preprint arXiv:2410.03859*.
6. Zhuo, Terry Yue, et al. "Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions." arXiv preprint arXiv:2406.15877 (2024).

Division of Responsibilities

Name	Responsibilities
Tingfeng Lan	Agent framework, tool integration, system testing
Yusen Wu	Problem framing, system debugging and evaluation, planner design and construct dataset
Yi Deng	Construct dataset, searching user cases and related papers